

Classification & Régression Logistique

Formation ML — Séance 3

3 octobre 2026

Rappel — Séance 2

- Un algorithme d'apprentissage a toujours 3 briques : ① modèle, ② fonction de coût, ③ optimisation
- On a construit une régression pour prédire une valeur **continue**
- La descente de gradient suit la pente pour minimiser le coût

Intuition — Aujourd'hui, la target n'est plus continue mais **binaire** : survécu ou non, spam ou non, malade ou non. Les 3 mêmes briques vont servir — mais leur contenu change en profondeur.

Au programme

1. Pourquoi une droite ne suffit plus
2. **Chapitre 1 — Vue d'ensemble** : on pose le modèle, on vérifie qu'il marche
3. **Chapitre 2 — Comment ça marche vraiment** : la dérivation complète
4. **Chapitre 3 — Philosophie** : frontière de décision, multiclasse, et au-delà

Cette séance suit volontairement la structure du cours de référence sur le sujet (Stanford CS109) : on pose d'abord le modèle et on observe qu'il fonctionne, **avant** de justifier rigoureusement pourquoi — plutôt que l'inverse.

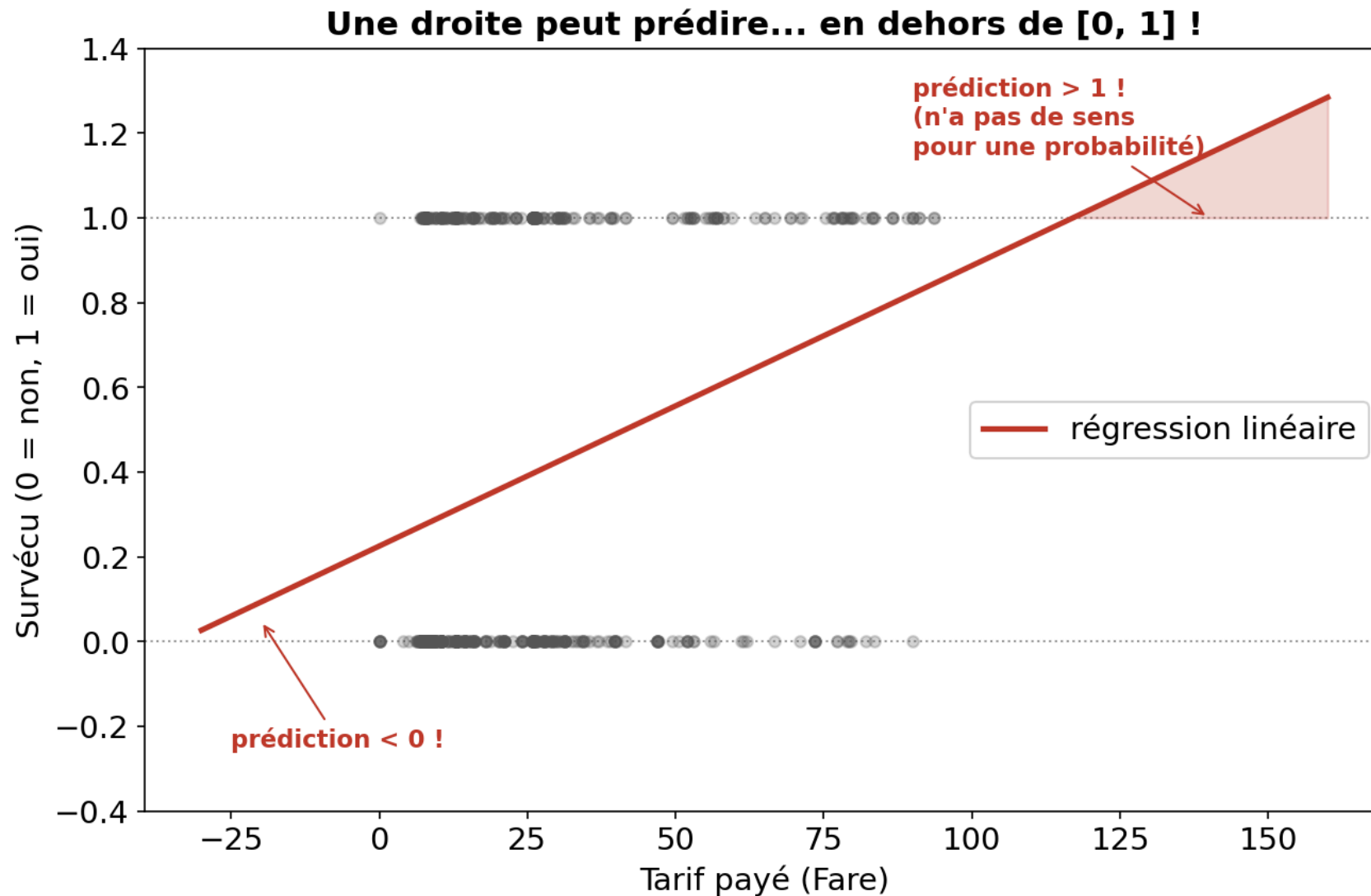
1. Pourquoi une droite ne suffit plus

Le nouveau problème : Titanic

- **Target** : **Survived** — 0 ou 1, pas une valeur continue
- On veut en réalité une **probabilité** : $P(\text{survie} \mid x)$
- Une probabilité doit rester dans $[0, 1]$

Attention — Rien n'empêche une droite $\hat{y} = wx + b$ de sortir de cet intervalle.

La preuve visuelle



Chapitre 1 : Vue d'ensemble

Et si, au lieu de chercher...
...on posait directement la bonne réponse ?

Puis on vérifie qu'elle marche.

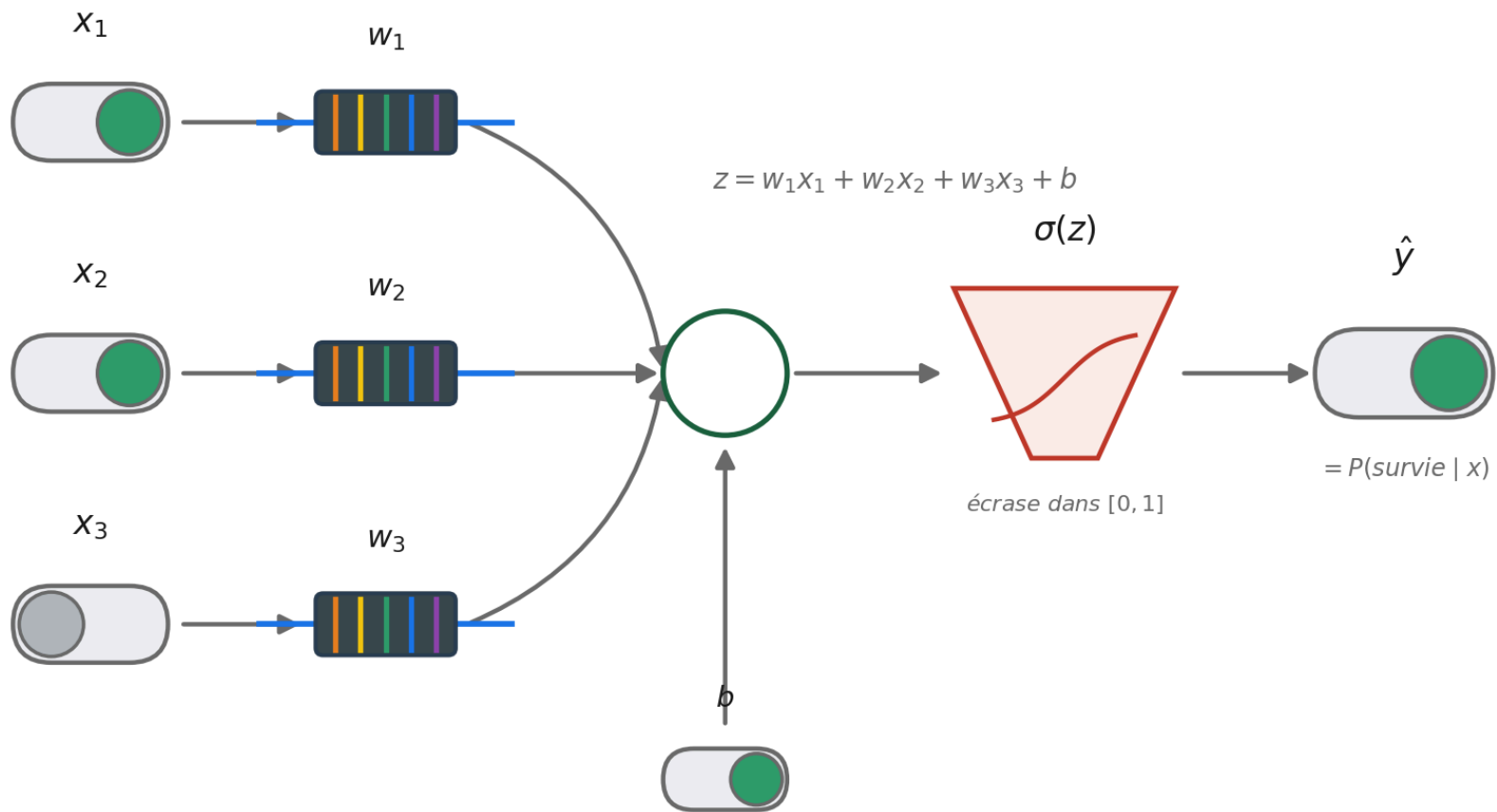
On pose le modèle directement

Plutôt que de le justifier étape par étape, on **pose** le modèle tout de suite — exactement comme on poserait une hypothèse de travail :

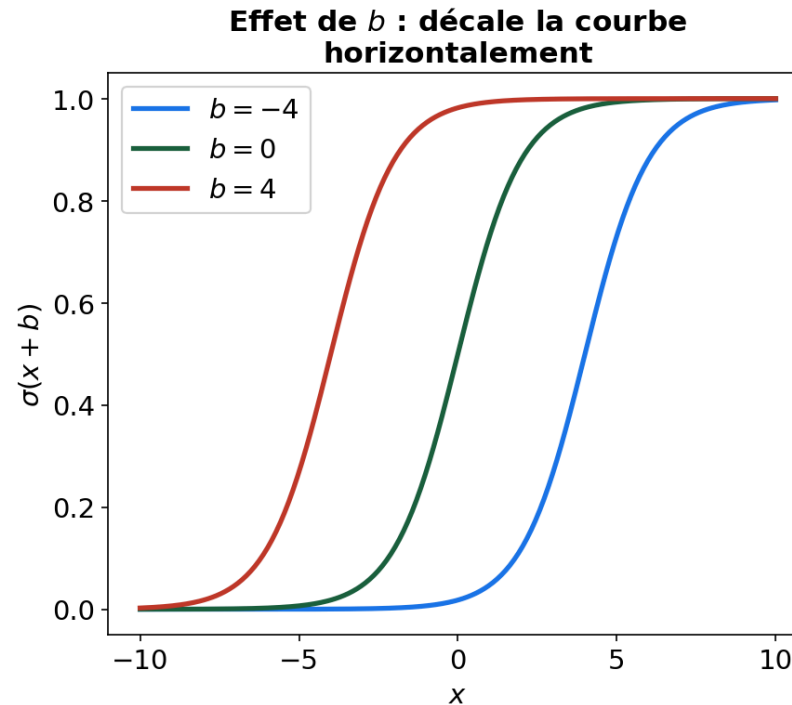
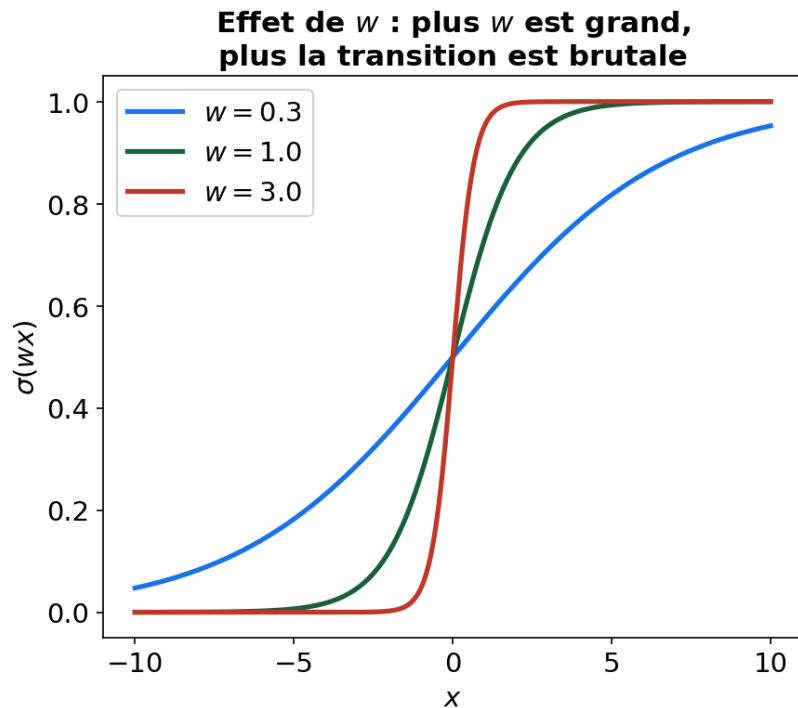
$$\hat{y} = \sigma(wx + b) \quad \text{où} \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Propriété de σ	Conséquence
$\sigma(0) = 0.5$	$z = 0$ est le point de bascule
Valeurs dans $]0, 1[$	jamais 0 ni 1 exactement — toujours une probabilité valide
$\sigma'(z) = \sigma(z)(1 - \sigma(z))$	dérivée simple — utile plus tard

Le schéma du modèle



L'effet de w et b sur la forme



À retenir — w contrôle la **brutalité** de la transition, b contrôle **où** se situe le point de bascule. Ce sont exactement les deux paramètres que l'optimisation va ajuster.

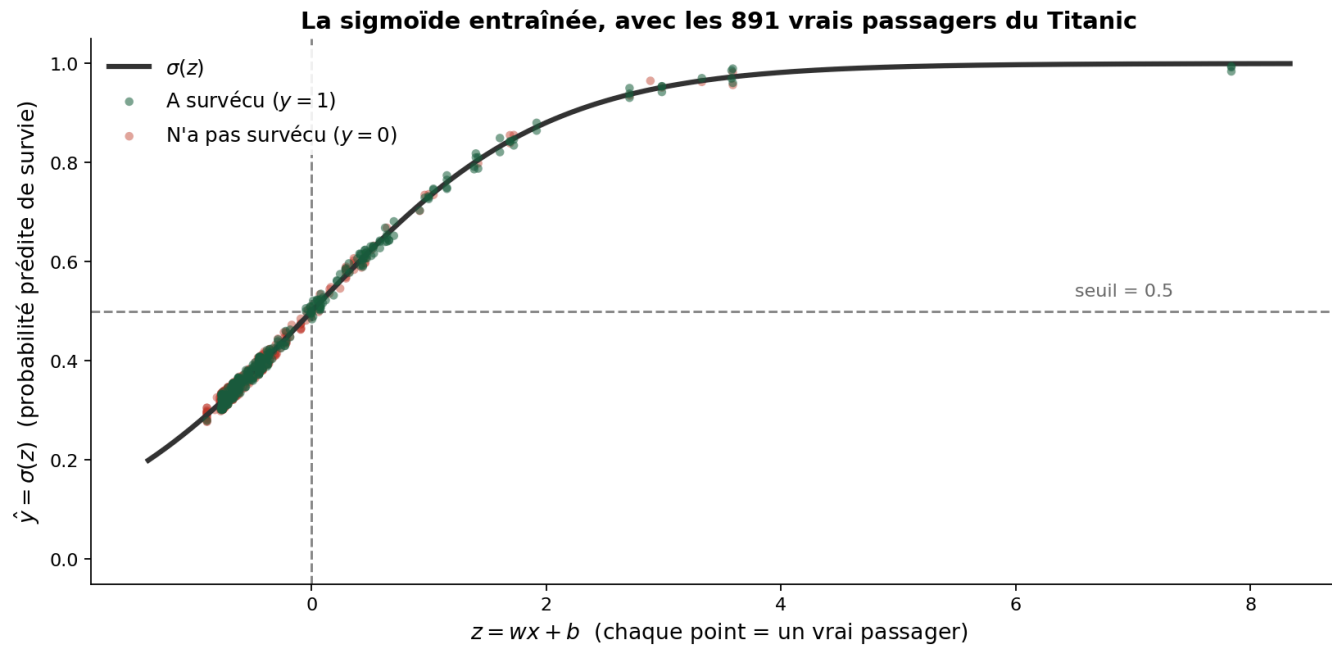
Est-ce que ça marche ? Un exemple réel

Sur de vraies données Titanic (**Fare** standardisé), on compare les prédictions **avant** et **après** apprentissage, pour un vrai passager :

Passager	Fare	Survived	(w, b) initial	$\hat{y}$	(w, b) entraîné	$\hat{y}$
W. Dodge (1ère classe)	81.86	1	(0.07, -0.05)	0.50	(0.85, -0.34)	0.60
L. Leonard (3ème classe)	0.00	0	(0.07, -0.05)	0.48	(0.85, -0.34)	0.29

À retenir — Avant entraînement, $\hat{y} \approx 0.5$ pour tout le monde — le modèle ne sait rien. Après, la prédiction se décale dans le **bon sens** pour chacun : plus haute pour le passager qui a survécu, plus basse pour celui qui n'a pas survécu.

Est-ce que ça marche ? Vue d'ensemble



À retenir — Chaque point est un **vrai** passager, placé sur la sigmoïde entraînée à son vrai z . Les points verts (survie) dominant à droite, les rouges (non-survie) dominant à gauche — le modèle capture un vrai signal, sans jamais être parfait : c'est normal, et on en reparlera en Séance 4.

D'où vient ce nom bizarre ?

« Régression logistique » pour de la **classification** — le nom est historiquement trompeur, et ça vaut le coup de savoir pourquoi.

- « **Logistique** » vient de la *fonction logistique*, publiée par **Verhulst en 1838** pour modéliser la croissance des populations — rien à voir, à l'origine, avec le Machine Learning
- Le terme « **logit** » (log de la cote) est popularisé dans les années 1940 par le statisticien **Joseph Berkson**
- « **Régression** » : on estime des coefficients (w, b) comme en régression linéaire — même si le résultat sert à classer, pas à prédire une valeur continue

Comment apprend-on w et b ? (aperçu)

On cherche les (w, b) qui rendent les données observées les **plus vraisemblables possible**. La justification précise de cette phrase attend le chapitre 2 — pour l'instant, on admet la règle de mise à jour, **identique** à celle de la Séance 2 :

$$w \leftarrow w - \alpha \frac{\partial \text{Log-loss}}{\partial w} \quad b \leftarrow b - \alpha \frac{\partial \text{Log-loss}}{\partial b}$$

À retenir — Même mécanique qu'en Séance 2 (avancer dans le sens opposé au gradient) — seule la fonction de coût change. C'est exactement ce que le tableau ci-dessus vient de montrer en action.

Chapitre 2 : Comment ça marche vraiment ?

D'où vient vraiment cette formule de coût ?

On ne choisit pas le log-loss au hasard parce qu'il "marche bien" — il découle directement d'un principe statistique : le **maximum de vraisemblance**.

Pour une observation, le modèle prédit $\hat{y} = P(y = 1 \mid x)$. La probabilité que le modèle attribue à ce qui s'est **réellement produit** est :

$$P(y \mid x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

(vérifie : si $y = 1$, ça vaut $\hat{y}$; si $y = 0$, ça vaut $1 - \hat{y}$)

De la vraisemblance au log-loss

On veut les paramètres qui **maximisent** la probabilité d'observer les vraies données — sur tout le dataset (probabilités indépendantes, donc on multiplie) :

$$L(w, b) = \prod_{i=1}^n \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

Attention — Multiplier des centaines de petites probabilités donne un nombre proche de zéro — problématique numériquement. On passe au **log** (transforme les produits en sommes, ne change pas où se situe le maximum).

Et enfin, minimiser plutôt que maximiser

$$\log L(w, b) = \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

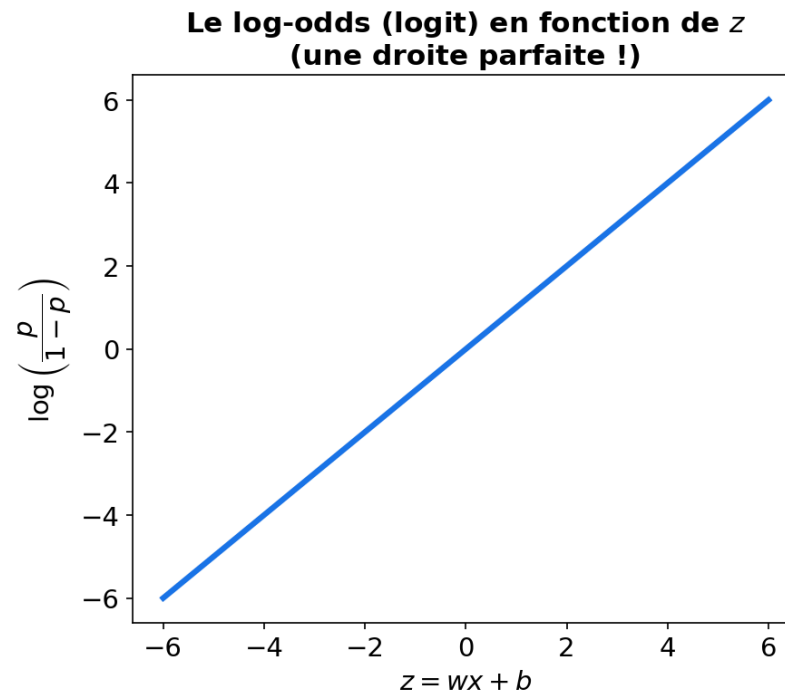
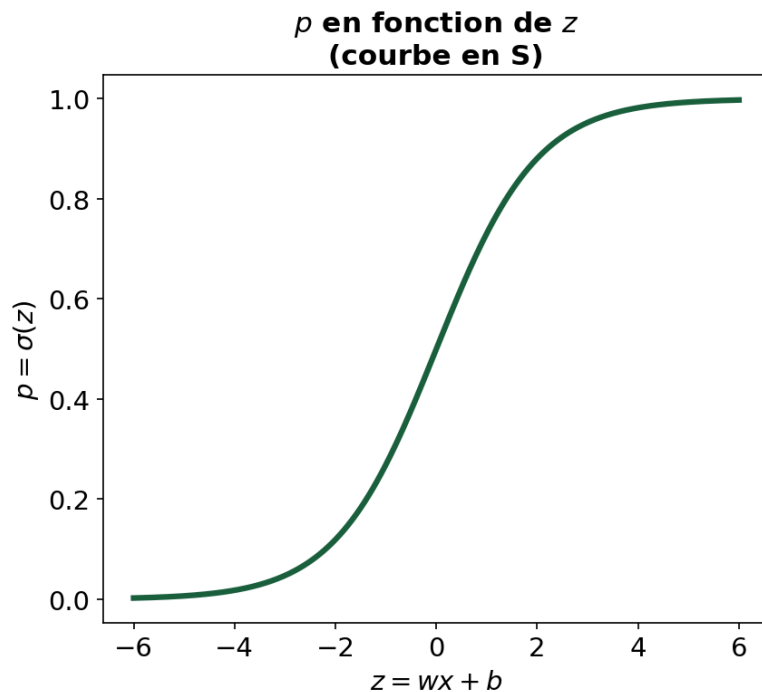
Par convention, l'optimisation **minimise** — on prend donc l'opposé :

$$\text{Log-loss}(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

À retenir — Ce n'est pas une formule arbitraire choisie pour ses bonnes propriétés — c'est la conséquence directe de "trouver les paramètres les plus vraisemblables au vu des données observées".

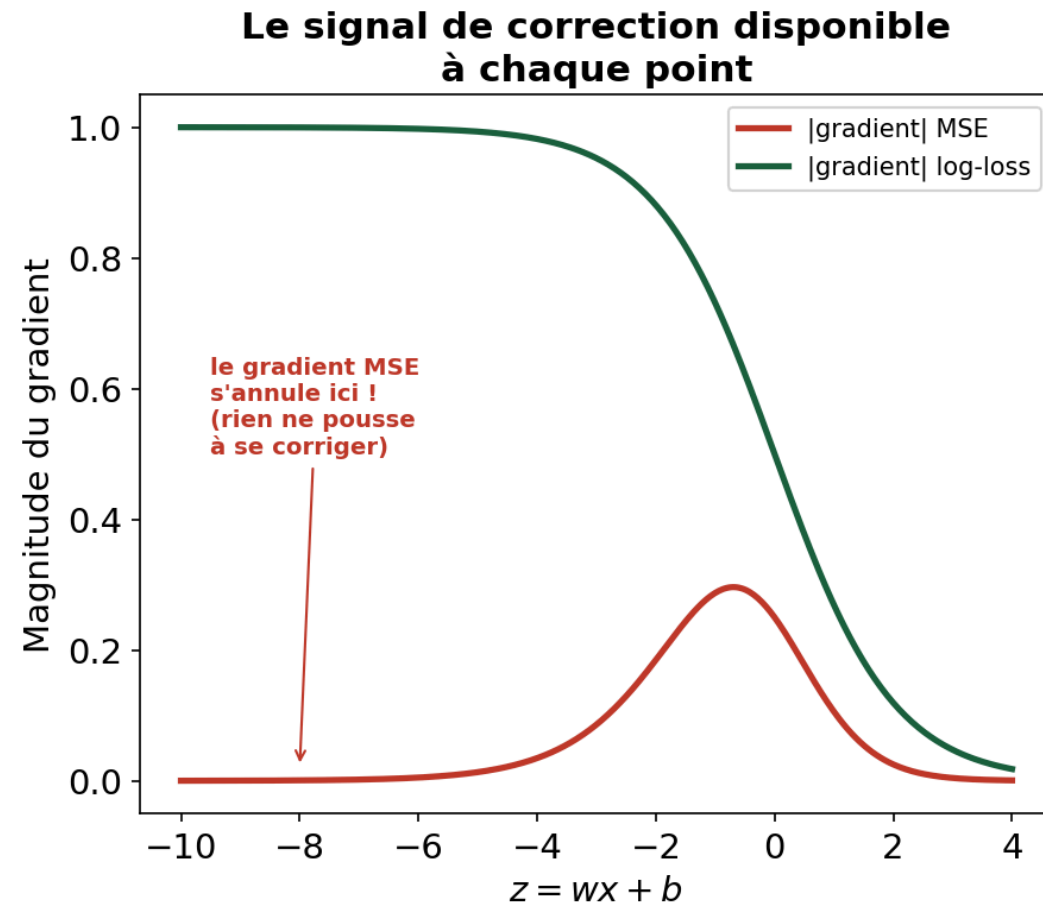
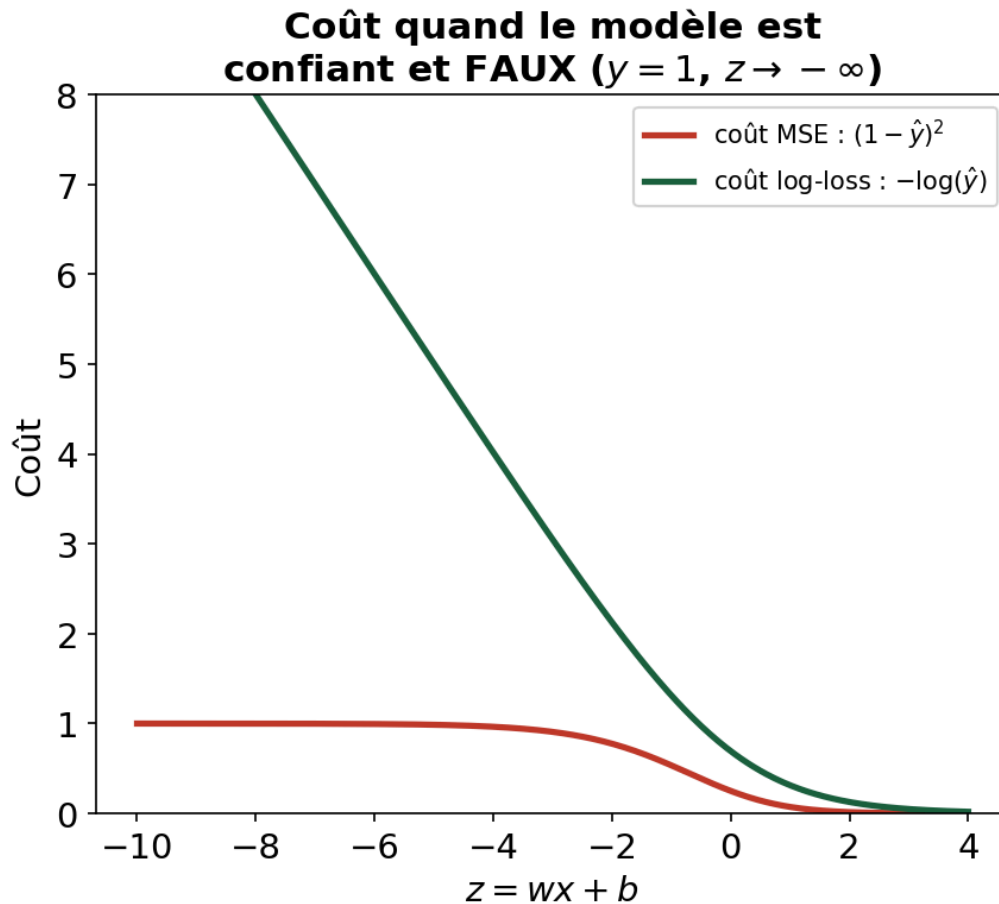
Un détour classique : la cote (odds)

Historiquement (et c'est l'origine du nom "logit", vu au chapitre 1), on justifie souvent σ par un autre chemin : modéliser le **log de la cote** $\log\left(\frac{p}{1-p}\right)$ de façon linéaire. En isolant p , on retombe exactement sur $\sigma(wx + b)$.

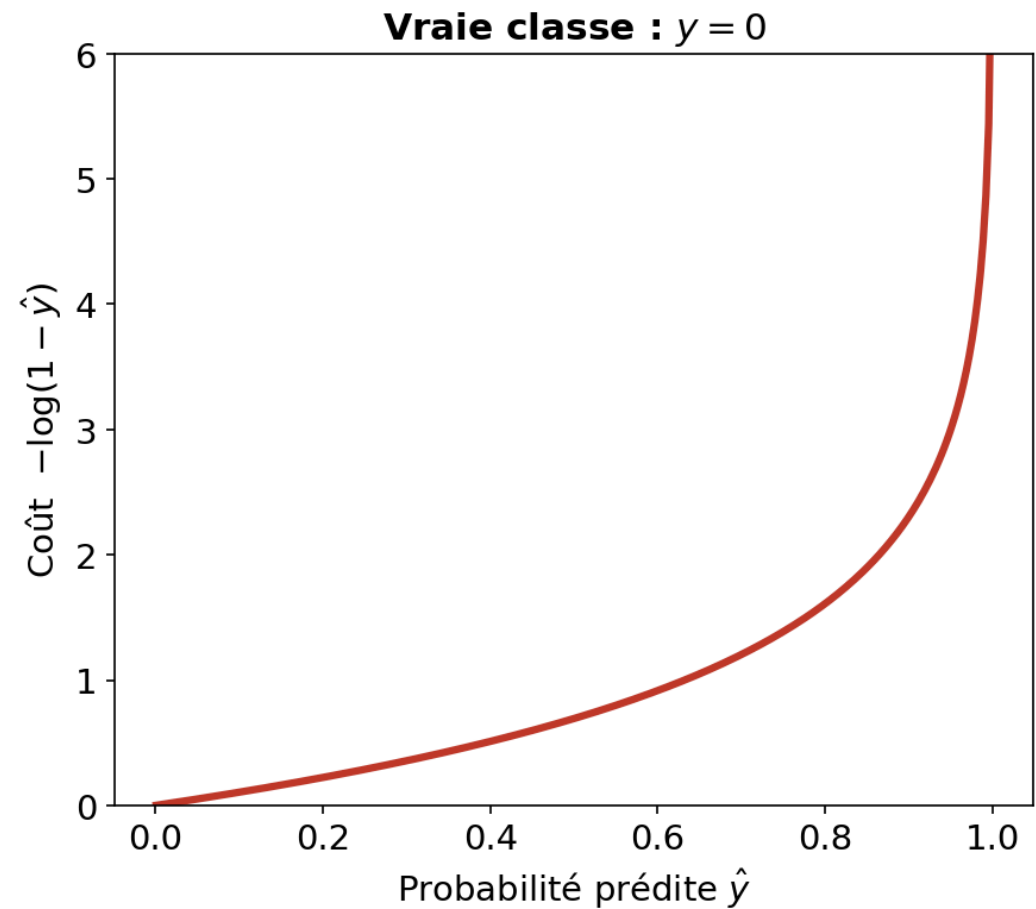
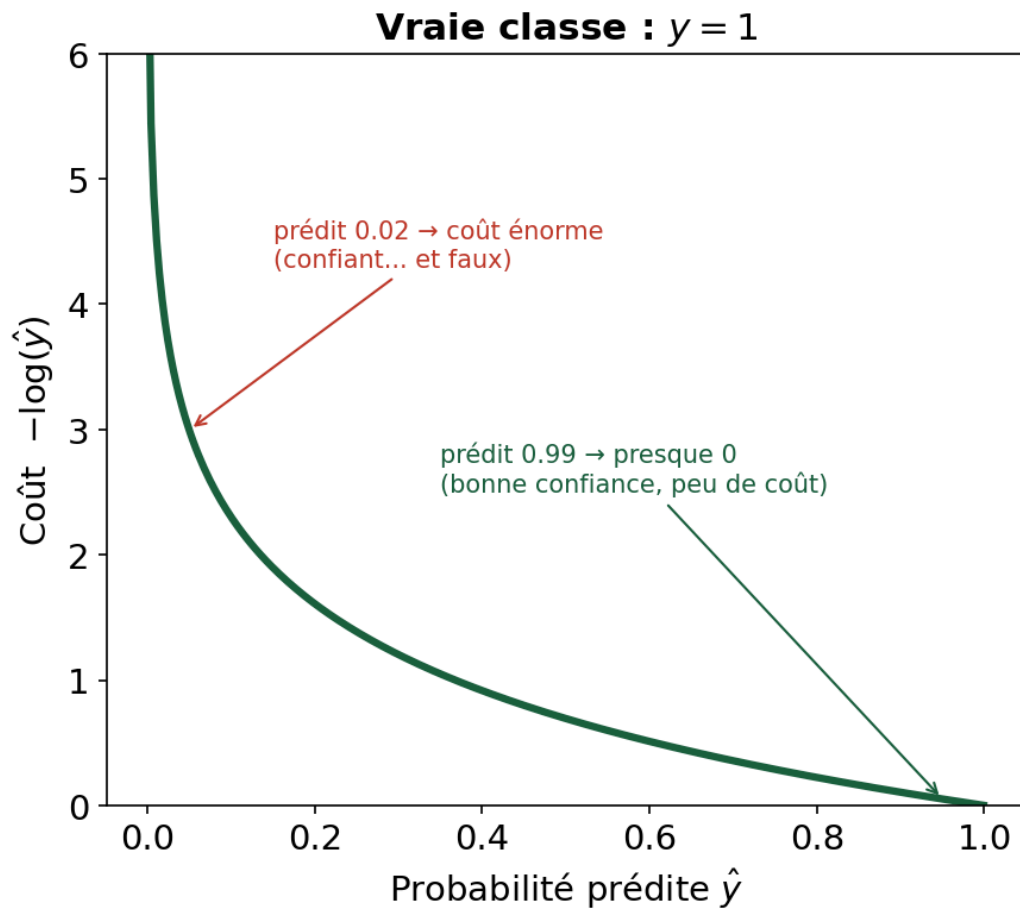


Pourquoi pas la MSE ? Le vrai problème

Voyons ce qui se passe quand le modèle est **confiant et faux** — $y = 1$ mais $\hat{y}$ proche de 0.



Visualiser le log-loss, terme par terme



Calcul à la main, sur 4 vraies lignes Titanic

Avec le modèle entraîné plus haut ($w \approx 0.846$, $b \approx -0.342$) :

Fare	Survived (y)	z	$\sigma(z)$	log-loss
21.08	0	-0.560	0.364	0.452
39.69	0	-0.262	0.435	0.571
11.13	1	-0.719	0.328	1.116
10.50	1	-0.729	0.325	1.123

Log-loss moyenne = $(0.452 + 0.571 + 1.116 + 1.123) / 4 = \mathbf{0.815}$

Attention — Les deux passagers ayant survécu (lignes 3-4) ont payé un tarif faible — le modèle leur attribue une **faible** probabilité de survie (~ 0.33), d'où un coût élevé : l'erreur du modèle, chiffrée précisément.

Comment obtient-on vraiment ce gradient ?

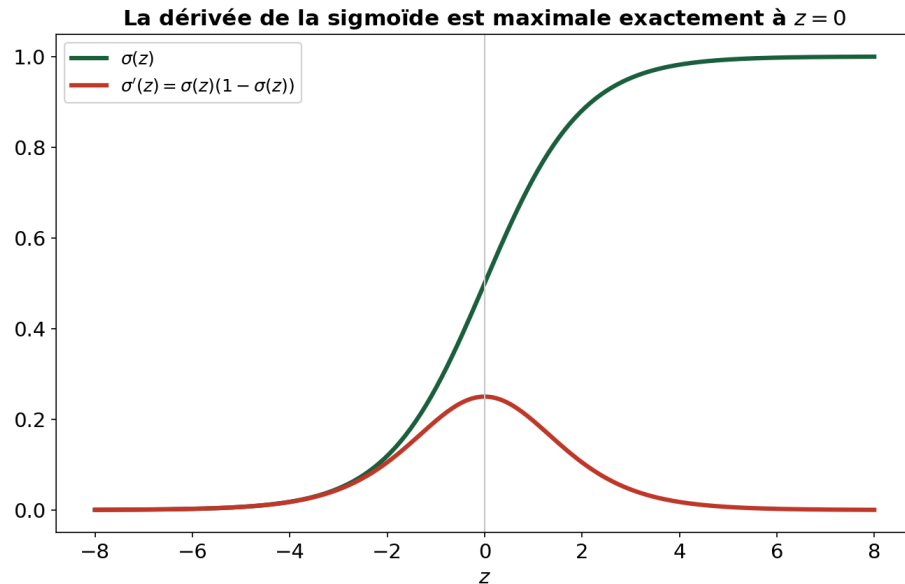
Ne pas admettre la formule du gradient — la construire

On va calculer $\frac{\partial \text{Log-loss}}{\partial w}$ par la règle de la chaîne, en 3 étapes : d'abord par rapport à $\hat{y}$, puis par rapport à z , puis par rapport à w . **Astuce** : on raisonne d'abord sur **une seule** observation — on sommera sur tout le dataset à la toute fin.

Étape 1 — dérivée du log-loss par rapport à $\hat{y}$ (pour une observation) :

$$\frac{\partial \text{coût}}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

La sigmoïde a une pente magnifique



utile à l'étape 2 →

Maximale exactement à $z = 0$ (valeur 0.25) — la sigmoïde y est la plus "sensible" à un changement de z .

Étape 2 — utiliser la dérivée de la sigmoïde

$$\frac{\partial \hat{y}}{\partial z} = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$$

déjà vu, slide précédente !

En combinant les étapes 1 et 2 par la règle de la chaîne :

$$\frac{\partial \text{coût}}{\partial z} = \frac{\partial \text{coût}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \hat{y} - y$$

ça se simplifie magnifiquement !

À retenir — Les termes $\hat{y}(1 - \hat{y})$ se simplifient élégamment avec le dénominateur — c'est loin d'être un hasard : c'est précisément **pourquoi** log-loss et sigmoïde sont faits pour aller ensemble.

Étape 3 — et enfin par rapport à w

Puisque $z = wx + b$, on a $\frac{\partial z}{\partial w} = x$ — règle de la chaîne, une dernière fois. Donc, sur tout le dataset :

$$\frac{\partial \text{Log-loss}}{\partial w} = \frac{1}{n} \sum_{i=1}^n x_i (\hat{y}_i - y_i)$$

$$\frac{\partial \text{Log-loss}}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

même forme qu'en régression linéaire !

À retenir — C'est **exactement** la même forme qu'en régression linéaire (erreur $\times$ feature) — et c'est précisément ce qu'on a branché, sans le justifier, dans la règle de mise à jour du chapitre 1.

Récapitulatif — la recette complète

1

Poser l'hypothèse — $\hat{y} = \sigma(wx + b)$, directement (chapitre 1).

2

Écrire la vraisemblance des données, puis passer au log $\rightarrow$ Log-loss (maximum de vraisemblance).

3

Dériver par la règle de la chaîne, une étape à la fois ($\hat{y} \rightarrow z \rightarrow w$) $\rightarrow$ gradient $= x(\hat{y} - y)$.

À retenir — Trois étapes, et rien d'autre. Toute la mécanique de la régression logistique tient dans cette recette — c'est elle qu'on retrouvera, démultipliée, dans un réseau de neurones entier.

Ce que w signifie concrètement

Puisque $\log(\text{cote}) = wx + b$ (vu dans le détour sur la cote), augmenter x de 1 unité **multiplie la cote par e^w** .

$$\text{nouvelle cote} = \text{ancienne cote} \times e^w$$

e^w s'appelle l'**odds ratio** — c'est la grandeur que les praticiens (épidémiologistes, data scientists) utilisent réellement pour interpréter un modèle logistique.

Sur de vraies données Titanic

En entraînant une régression logistique sur **Fare** (standardisé) :

$$w \approx 0.846 \quad b \approx -0.342$$

$$e^w = e^{0.846} \approx 2.330$$

À retenir — Chaque augmentation d'un écart-type du tarif payé **multiplie par ~2.33 la cote de survie** — plus qu'un doublement. C'est une phrase qu'on peut dire à quelqu'un qui ne fait pas de ML, contrairement à "le coefficient vaut 0.846".

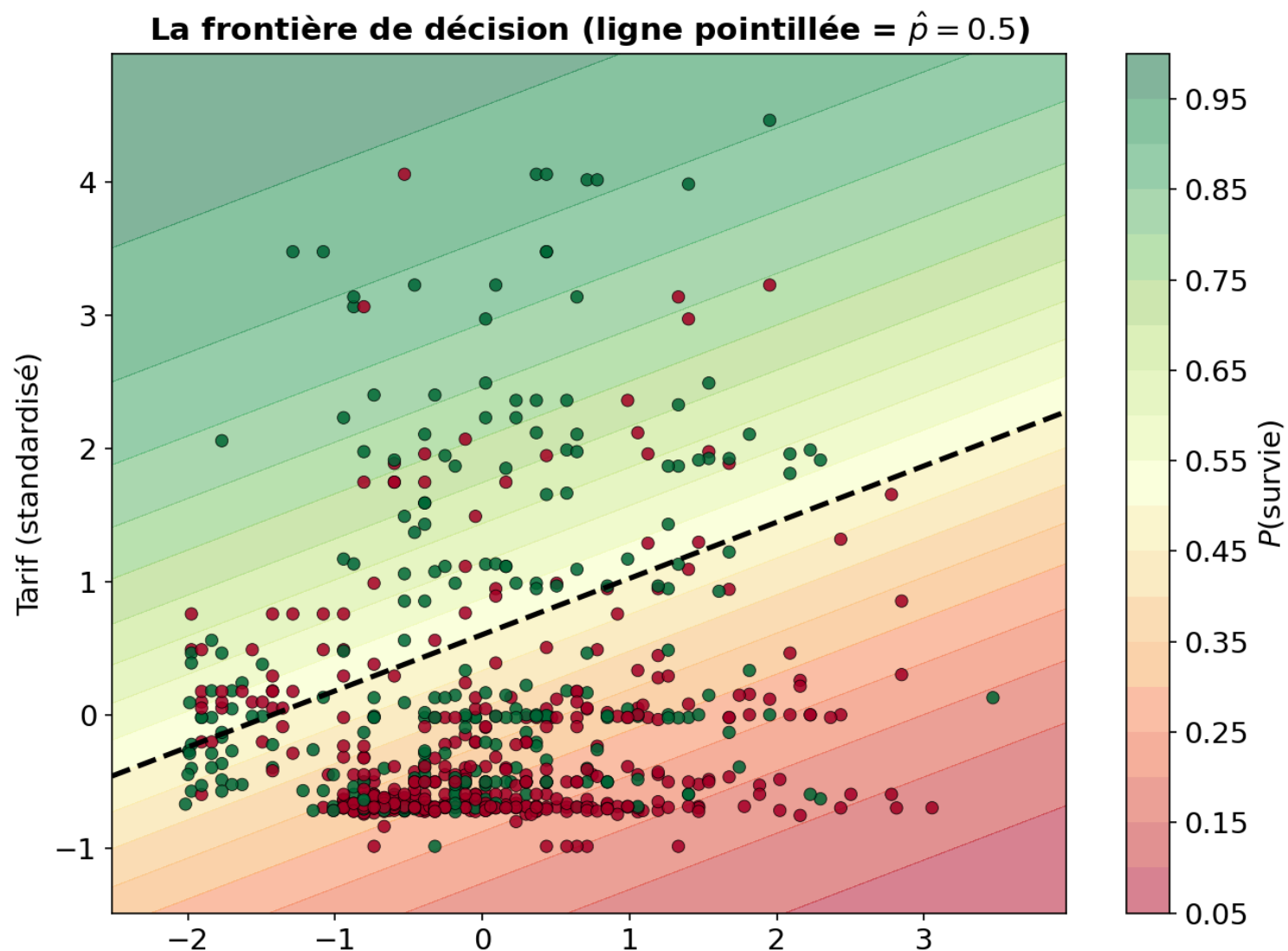
Chapitre 3 : Philosophie

Ce que "décider" veut dire géométriquement

Le seuil $\hat{y} = 0.5$ correspond à $z = wx + b = 0$ — une **ligne** (ou un hyperplan, en dimension supérieure) qui sépare l'espace des features en deux régions.

Avec deux features (**Age** , **Fare**), on peut la visualiser directement.

La frontière, en pratique



Et s'il y a plus de 2 classes ?

La régression logistique telle qu'on l'a construite est **binaire**. Pour K classes (ex : type de fleur, catégorie de produit), on généralise avec la fonction **softmax**, qui étend la sigmoïde à plusieurs sorties.

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Attention — Chaque classe a son propre $z_k = w_k x + b_k$. Le softmax garantit que toutes les probabilités somment à 1. Hors scope détaillé de cette séance.

Le lien avec les réseaux de neurones

Regarde à nouveau le schéma du chapitre 1 : entrée $\rightarrow$ combinaison linéaire $\rightarrow$ fonction d'activation. C'est très exactement un **neurone artificiel**.

- Une régression logistique = **un seul neurone**, avec σ comme fonction d'activation
- Le **deep learning**, dans son principe, empile des milliers de ces neurones en couches successives
- Même brique ①②③ (modèle, coût, optimisation) — juste beaucoup plus de neurones, et une fonction de coût adaptée

À retenir — Tu ne viens pas seulement d'apprendre un algorithme de classification — tu viens de construire, à la main, la brique de base de tous les réseaux de neurones.

Débrief

Ce qu'on retient

Brique	Régression (S2)	Classification (S3)
① Modèle	$\hat{y} = wx + b$	$\hat{y} = \sigma(wx + b)$ — posé directement, justifié au chapitre 2
② Coût	MSE	Log-loss, dérivé du maximum de vraisemblance
③ Optimisation	Descente de gradient	Descente de gradient (mécanique identique)

À retenir — Rien n'est arbitraire dans ce qu'on a construit aujourd'hui : on a vérifié que le modèle marche avant de prouver pourquoi — l'ordre inverse de ce qu'on fait d'habitude en cours, mais celui qu'on retient le mieux.

→ **Séance 4 : généralisation, validation, prétraitement**

Merci !

Questions ?

Canal WhatsApp — questions ML